pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Request Logs and Monitoring

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: request logs and monitoring

Today

1. What to log, and what never to log
2. log_middleware on the bond service
3. On a platform, standard output
4. A month of requests by path
5. Percentiles, and the index that differs
6. Alerts by rule
7. An outage, from the record
8. What Vercel kept of the course's requests

Two logs

Our own

10 requests, through the test client

on a fake clock: the same every run

A month, synthetic

19,664 requests, 2026-09-03 to 2026-10-02

three features planted for today

A deployed service's only record of what it did is what it wrote down at the time.

What a log's numbers are called

Requests, latency in milliseconds, an error rate: about the service, never about a bond.

Allowed names

requests

p95 latency in ms

error rate

Never called

performance of the desk

P&L

a health score of a market

A request that took 12 ms says nothing about the bond it asked about, or the market.

What to log, and what never to log

Seven columns

time_utc: UTC, with milliseconds

request_id: 16 hexadecimal characters

path: without the query string

status: 200, 404, 422, 500

latency_ms

bytes_out

refused: the guardrail, counted

Never

a request's body: the bond, the question

a query value

a header's value

the client's address

a response body (a 422 echoes the body)

The stated exception: day 13's model keeps a separate log of its inputs, for drift, and nothing about the sender.

log_middleware on the bond service

```
wall, perf = deploykit.fake_clock("2026-10-02T14:00:00", 12.0)
app = deploykit.log_middleware(deploykit.bond_app(), write_line, clock=perf, wall=wall)
```

Ten requests through the test client: [200, 200, 200, 200, 200, 200, 422, 404, 200, 200]

```
{"time_utc": "2026-10-02T14:00:00.000Z", "request_id": "39bcadea4c81a848", "path": "/health",
  "status": 200, "latency_ms": 12.0, "bytes_out": 44, "refused": false}
```

7

columns in every line, and nothing else

no

client's name or yield in a line

2

sinks: a file here, print on a platform

The client's name in the line: no; the yield 5.06 in the line: no.

On a platform, the log goes to standard output

A function's disk is not a log store

1. Read-only, apart from /tmp scratch space in one running copy
2. Copies come and go: archived when idle, many at once under load
3. What a function prints is kept, grouped by request

Vercel Functions: a read-only file system, writable /tmp up to 500 MB (vercel.com/docs/functions/runtimes, read 2026-10-12).

Vercel Hobby: runtime logs kept for 1 hour; Pro 1 day (vercel.com/docs/logs/runtime, read 2026-10-12).

Vercel runtime logs: up to 256 lines and 1 MB a request, 256 KB a line (vercel.com/docs/logs/runtime, read 2026-10-12).

Locally a file, on a platform standard output: the same log_middleware, two sinks.

A month of requests, by path

path	requests	errors_4xx	errors_5xx	refused	p50_ms	p95_ms	mean_ms
/accrued	1338	16	2	0	9.0	19.2	10.0
/ask	1370	182	1	116	27.6	58.0	30.9
/brief	1915	0	1	0	24.2	49.3	26.5
/checks	1003	0	1	0	17.7	36.4	19.6
/dv01	4625	53	3	0	12.3	28.9	15.3
/health	2935	0	0	0	3.0	6.3	3.4
/price	4971	305	3	0	10.7	23.4	12.0
/yield	1507	18	1	0	12.7	26.5	14.2

19,664 requests. Percentiles, not the mean: a few slow requests pull a mean up and say nothing about the typical one.

Percentiles: name the method

19.05 ms

linear: NumPy, pandas, PERCENTILE.INC

19.95 ms

PERCENTILE.EXC's rule

20.00 ms

by index: sorted(x)[int(0.95 * n)]

The p95 of 20 hand-made latencies, 1 ms to 20 ms: three methods, three numbers.

The month's /dv01 requests

All three give 28.90 ms: the latencies repeat at 0.1 ms, so the methods land on equal values.

The course uses linear everywhere, and log_summary's docstring says so.

Alerts by rule

```
days: 30; alerts: 2
2026-09-17: 605 requests, p95 101.2 ms, error share 2.1%
2026-09-24: 851 requests, p95 32.5 ms, error share 29.8%
on the other days: p95 at most 37.7 ms, error share at most 2.9%
```

154.0 ms

/dv01 p95 on the slow day

61.8%

/price error share, burst day

14:00

the hour of the 422 burst, UTC

422s on /price on 2026-09-24, by UTC hour: {13: 1, 14: 240, 19: 1}

A status page is the same rule, published for a service's users.

Why a request log and a status page

Fastly, "Summary of June 8 outage", 8 June 2021

85% of the network returned errors; the disruption began at 09:47 UTC

"We detected the disruption within one minute"

"Within 49 minutes, 95% of our network was operating as normal."

A bug from a deployment begun May 12, "triggered by a valid customer configuration change"

Source: www.fastly.com/blog/summary-of-june-8-outage, read 2026-10-12

Seen within a minute, on one clock; and a deploy that passes its tests is not the end of watching it.

What Vercel kept of the course's requests

What the platform printed, masked

```
Recorded 2026-10-12 on the course's own account; account names, URLs, and ids masked to `student`.
> vercel logs --project pfi-bond-service --since 30m
TIME         HOST                          LEVEL
12:16:31.93  student.vercel.app  info   ? GET /health
12:16:31.68  student.vercel.app  info   ? POST /price
12:16:31.38  student.vercel.app  info   ? GET /status
```

Kept with no code of yours: time, method, path, status, host, deployment, ids, the query string.

Not in the JSON: a latency or duration field. message empty: the app prints nothing.

The ten statuses on Vercel: the same as the test client's.

Hobby keeps an hour. Your own latency_ms column is what makes a p95 possible.

Excel and Git

Excel side by side (tested in Excel)

`=COUNTIFS(log!C2:C19665,"/dv01")`	4,625
`=COUNTIFS(log!C2:C19665,"/price",log!D2:D19665,">=400",log!D2:D19665,"<500")`	305
`=AVERAGEIFS(log!E2:E19665,log!C2:C19665,"/dv01")`	15.2804
`=PERCENTILE.INC(FILTER(log!E2:E19665,log!C2:C19665="/dv01"),0.95)`	28.90
`=PERCENTILE.INC(hand!A2:A21,0.95) / .EXC / SMALL(...,INT(0.95*n)+1)`	19.05 / 19.95 / 20.00

```
- "p95_limit_ms": 60.0,
+ "p95_limit_ms": 35.0,
```

A p95 limit of 35 ms instead of 60: days alerting go from 2 to 11. Read the diff before you commit.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: sorted(x)[int(0.95 * n)] agrees on the month and returns 20.00 on 1 to 20 ms, where linear gives 19.05.

Code that runs is not the same as code that is right.

What you can do now

- Keep seven columns; never a body, query value, header, or address
- Log every request with a middleware: a file here, print on a platform
- Summarise a month by path, by percentiles
- Take a p95 the way PERCENTILE.INC does, and name the method
- Find a slow day and a burst of errors by rule

Week 3, Day 3 Model files and drift